

Capstone Project

Name: Hitaansh Maheshwary
Batch: SDET Playwright(Batch 1)

Trainer: Mr. Piyush Gupta

Playwright Capstone Project

Application URL: <https://practicesoftwaretesting.com>

Objective

Automate a realistic end-to-end e-commerce workflow using Playwright.

This capstone project is designed to evaluate:

- Playwright fundamentals
- Locator strategy
- Assertions
- Synchronization handling
- Dynamic UI handling
- Reusability
- Automation stability
- Debugging understanding
- Real-world automation thinking

Important Instructions

Participants SHOULD:

- use meaningful assertions
- write reusable code
- handle dynamic UI properly
- use stable locator strategies
- maintain clean code structure

Participants SHOULD NOT:

- use unnecessary `waitForTimeout()`
- write everything in one file
- use brittle XPath indexes everywhere
- duplicate locator logic repeatedly
- hardcode validation values unnecessarily

Capstone Tasks

Task 1 - Homepage Validation

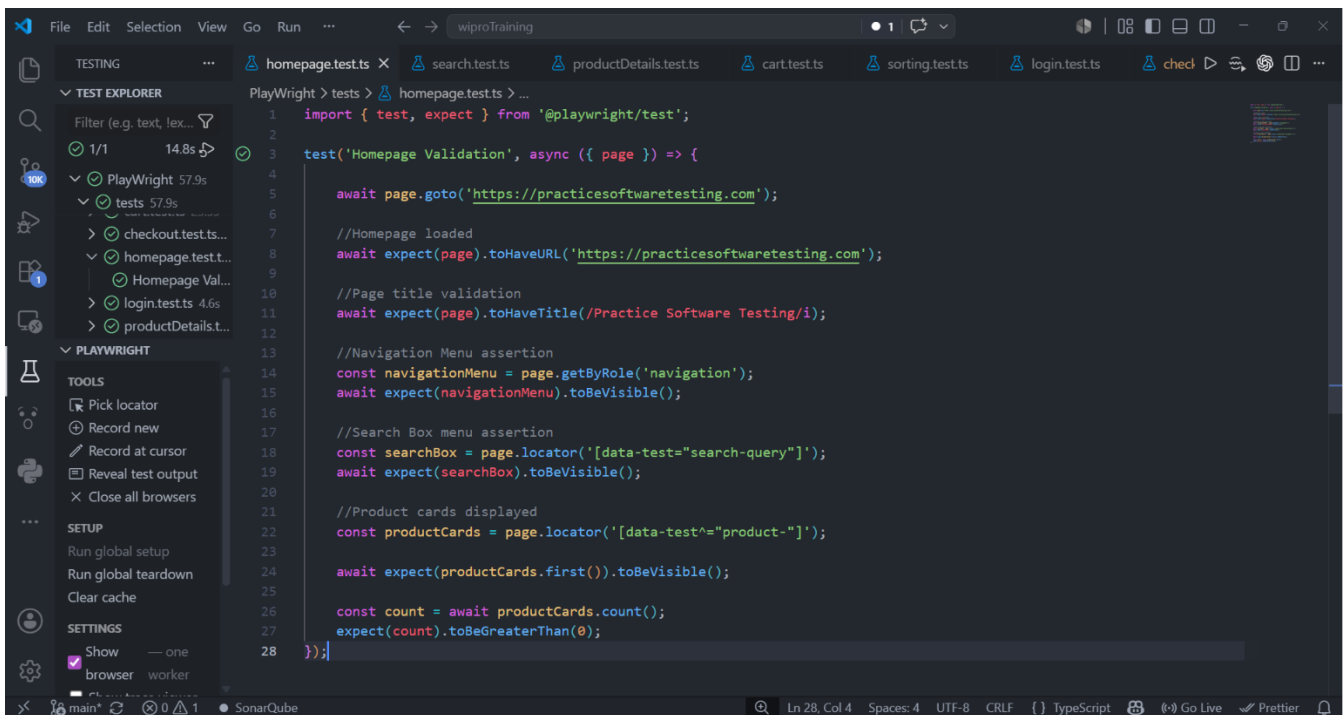
Requirements

Automate the following validations:

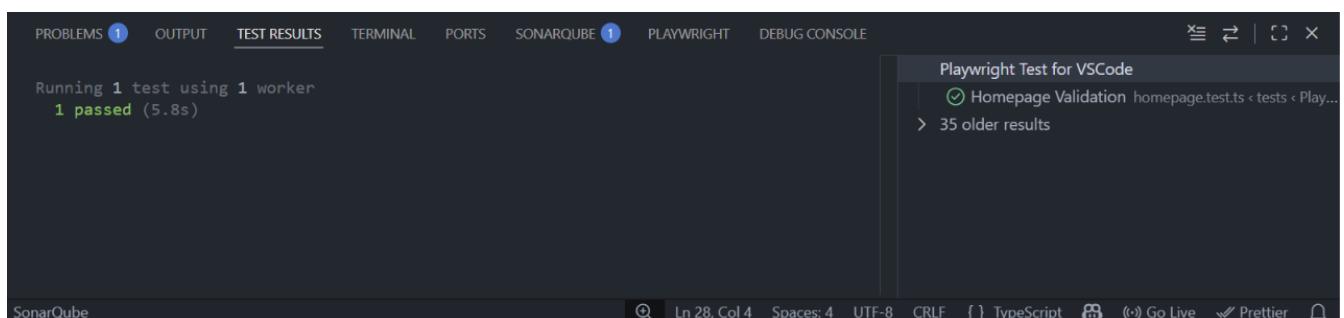
Validate:

- application homepage loads successfully
- page title validation
- main navigation menu visibility
- search bar visibility
- product cards are displayed

Screenshot of Code:



Screenshot of Output/Terminal:



Task 2 - Search Functionality

Requirements

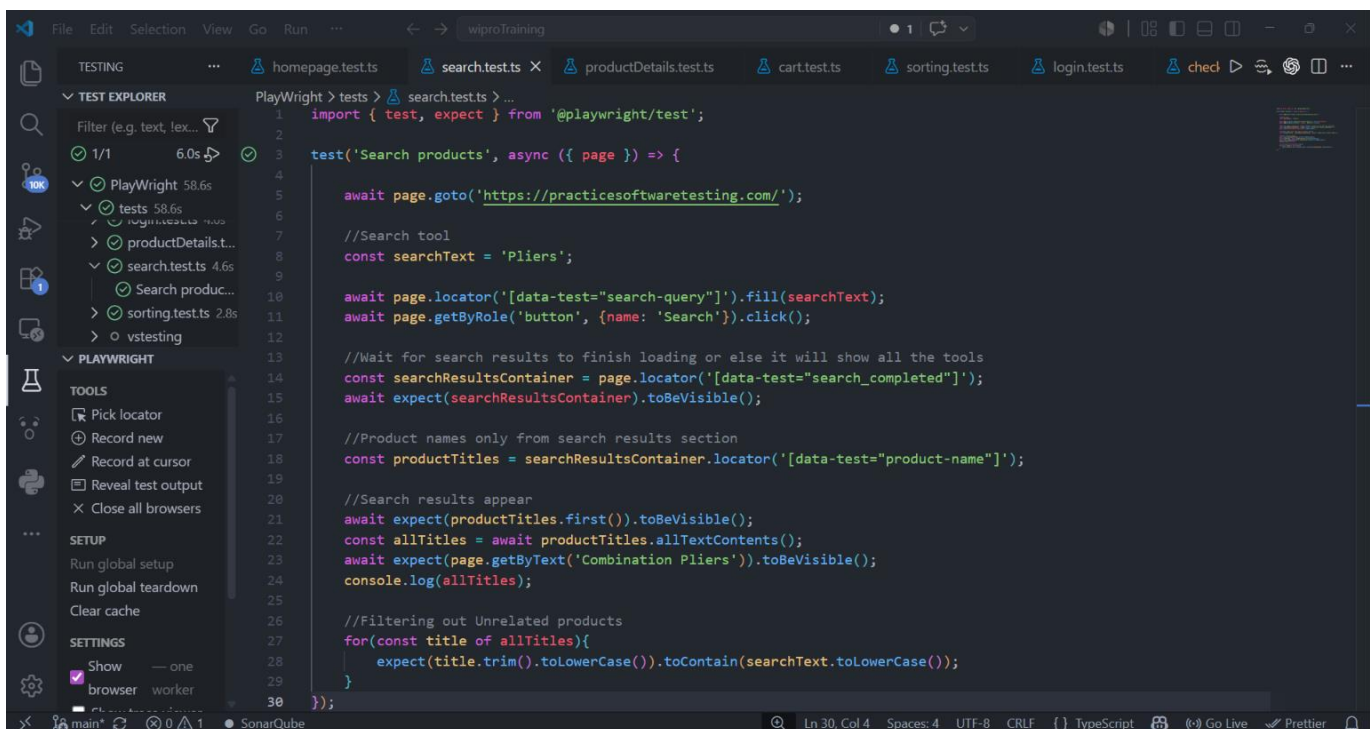
Search for a product dynamically.

Example: Hammer

Validate:

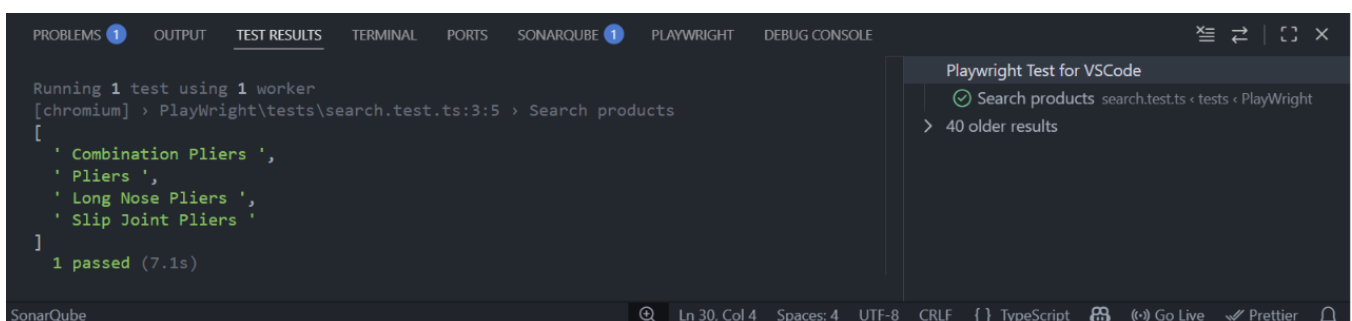
- search results appear correctly
- searched product is visible
- unrelated products are filtered out

Screenshot of Code:



```
1 import { test, expect } from '@playwright/test';
2
3 test('Search products', async ({ page }) => {
4
5     await page.goto('https://practicesoftwaretesting.com/');
6
7     //Search tool
8     const searchText = 'Pliers';
9
10    await page.locator('[data-test="search-query"]').fill(searchText);
11    await page.getByRole('button', {name: 'Search'}).click();
12
13    //Wait for search results to finish loading or else it will show all the tools
14    const searchResultsContainer = page.locator('[data-test="search_completed"]');
15    await expect(searchResultsContainer).toBeVisible();
16
17    //Product names only from search results section
18    const productTitles = searchResultsContainer.locator('[data-test="product-name"]');
19
20    //Search results appear
21    await expect(productTitles.first()).toBeVisible();
22    const allTitles = await productTitles.allTextContents();
23    await expect(page.getByText('Combination Pliers')).toBeVisible();
24    console.log(allTitles);
25
26    //Filtering out Unrelated products
27    for(const title of allTitles){
28        expect(title.trim().toLowerCase()).toContain(searchText.toLowerCase());
29    }
30 });
```

Screenshot of Output/Terminal:



```
Running 1 test using 1 worker
[chromium] > Playwright\tests\search.test.ts:3:5 > Search products
[
  'Combination Pliers ',
  'Pliers ',
  'Long Nose Pliers ',
  'Slip Joint Pliers '
]
1 passed (7.1s)
```

Sort

Price Range



Search

 X Search

Filters

By category:

- ☐ Hand Tools
 - ☐ Hammer
 - ☐ Hand Saw
 - ☐ Wrench
 - ☐ Screwdriver
 - ☐ Pliers
 - ☐ Chisels
 - ☐ Measures

Searched for: Pliers

4 products found for 'Pliers'



Combination Pliers

CO₂ A B C D E

\$14.15



Pliers

CO₂ A B C D E

\$12.01



Long Nose Pliers

CO₂ A B C D E

Out of stock

\$14.24



Task 3 - Product Details Validation

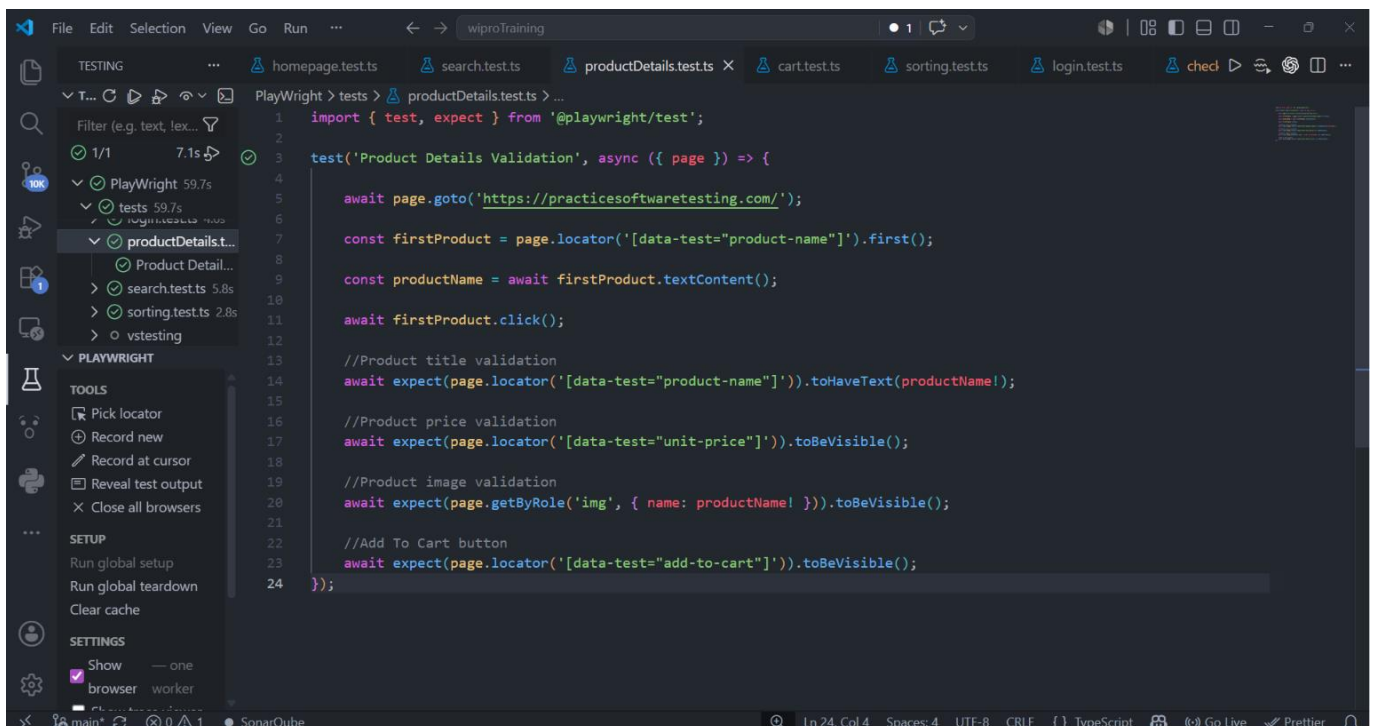
Requirements

Open a product details page.

Validate:

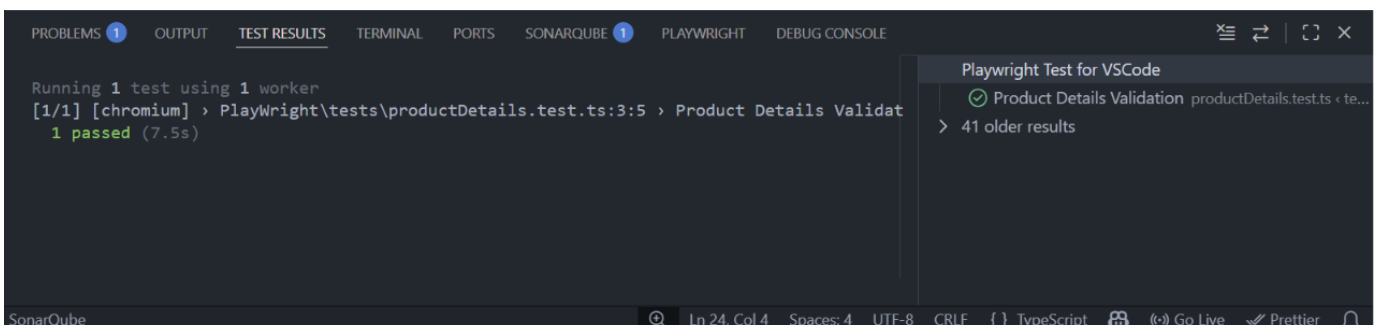
- product title
- product price
- product image visibility
- add-to-cart button visibility

Screenshot of Code:



```
1 import { test, expect } from '@playwright/test';
2
3 test('Product Details Validation', async ({ page }) => {
4
5     await page.goto('https://practicesoftwaretesting.com/');
6
7     const firstProduct = page.locator('[data-test="product-name"]').first();
8
9     const productName = await firstProduct.textContent();
10
11     await firstProduct.click();
12
13     //Product title validation
14     await expect(page.locator('[data-test="product-name"]').first()).toHaveText(productName!);
15
16     //Product price validation
17     await expect(page.locator('[data-test="unit-price"]').first()).toBeVisible();
18
19     //Product image validation
20     await expect(page.getByRole('img', { name: productName! })).toBeVisible();
21
22     //Add To Cart button
23     await expect(page.locator('[data-test="add-to-cart"]').first()).toBeVisible();
24 });
```

Screenshot of Output/Terminal:



```
Running 1 test using 1 worker
[1/1] [chromium] > Playwright\tests\productDetails.test.ts:3:5 > Product Details Validat
1 passed (7.5s)
```

Task 4 - Cart Functionality

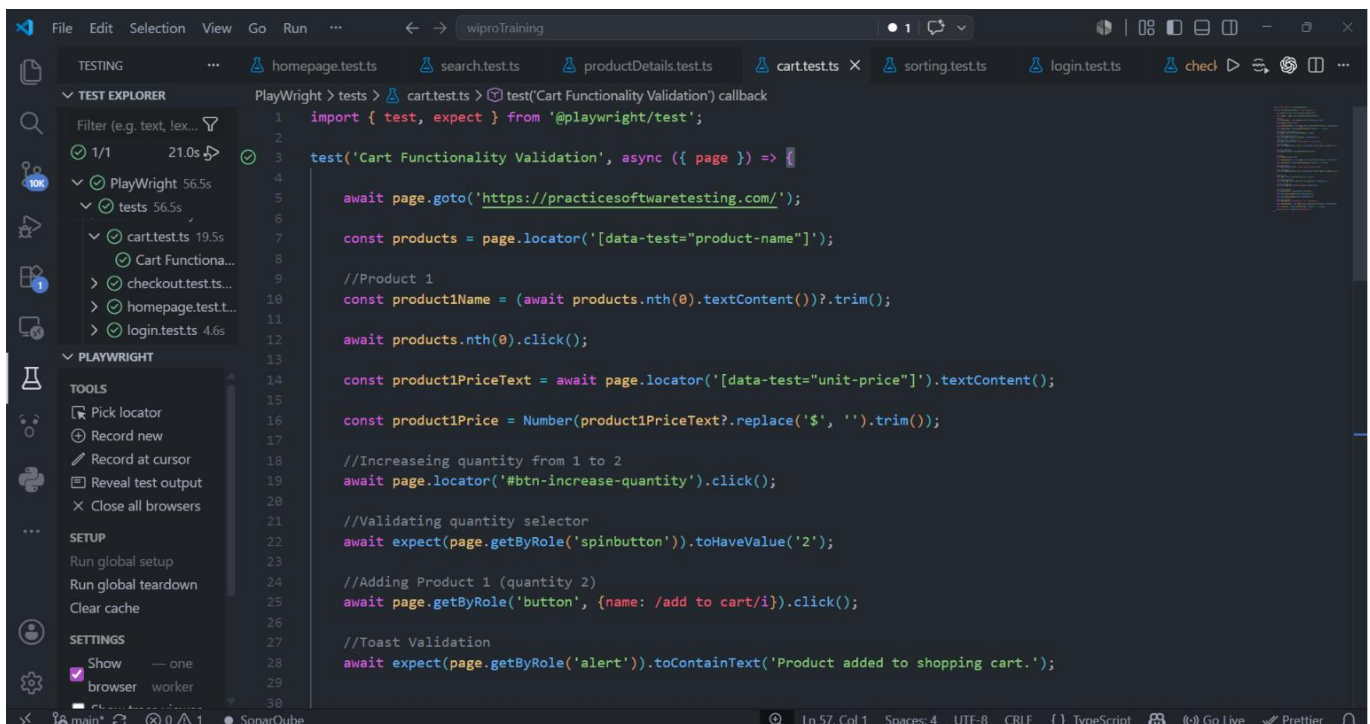
Requirements

Add at least TWO products to the cart. One product should have at least 2 quantities.

Validate:

- cart badge count updates correctly
- correct products appear in cart
- quantity validations
- subtotal validations

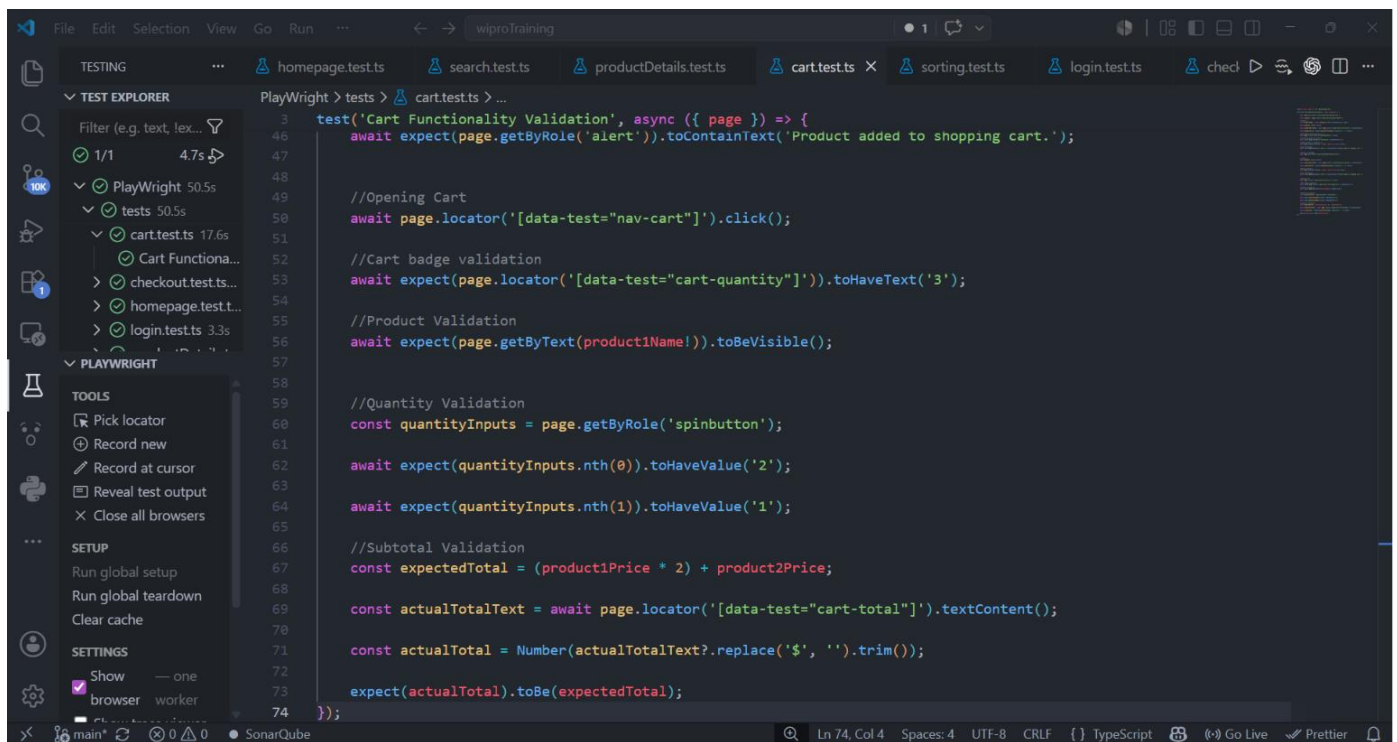
Screenshot of Code:



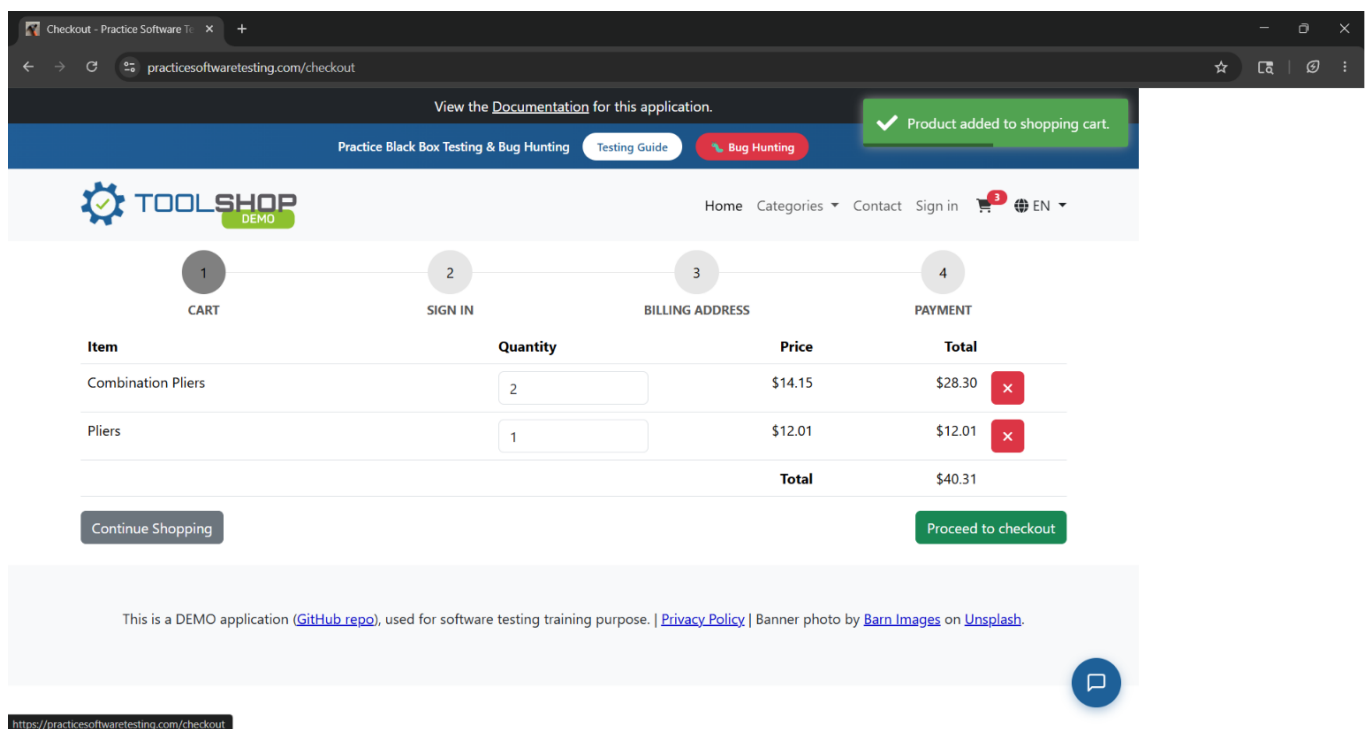
The screenshot shows a VS Code editor with a Playwright test file named `cart.test.ts`. The test is titled `test('Cart Functionality Validation', async ({ page }) => {`. The code includes the following steps:

- Import `test` and `expect` from `@playwright/test`.
- Navigate to `https://practicesoftwaretesting.com/`.
- Locate the product name and click it.
- Locate the unit price and convert it to a number.
- Click the 'btn-increase-quantity' button.
- Validate the quantity selector to have a value of '2'.
- Click the 'add to cart' button.
- Validate the toast message: 'Product added to shopping cart.'

```
30
31 //Return to Homepage
32 await page.goto('https://practicesoftwaretesting.com/');
33
34
35 //Product 2
36 await products.nth(1).click();
37
38 const product2PriceText = await page.locator('[data-test="unit-price"]').textContent();
39
40 const product2Price = Number(product2PriceText?.replace('$', '').trim());
41
42 //Adding Product 2
43 await page.getByRole('button', {name: /add to cart/i}).click();
44
45 //Toast Validation
46 await expect(page.getByRole('alert')).toContainText('Product added to shopping cart.');
```

Screenshot of Output/Terminal:



Task 5 - Sorting Validation

Requirements

Validate at least ONE sorting functionality.

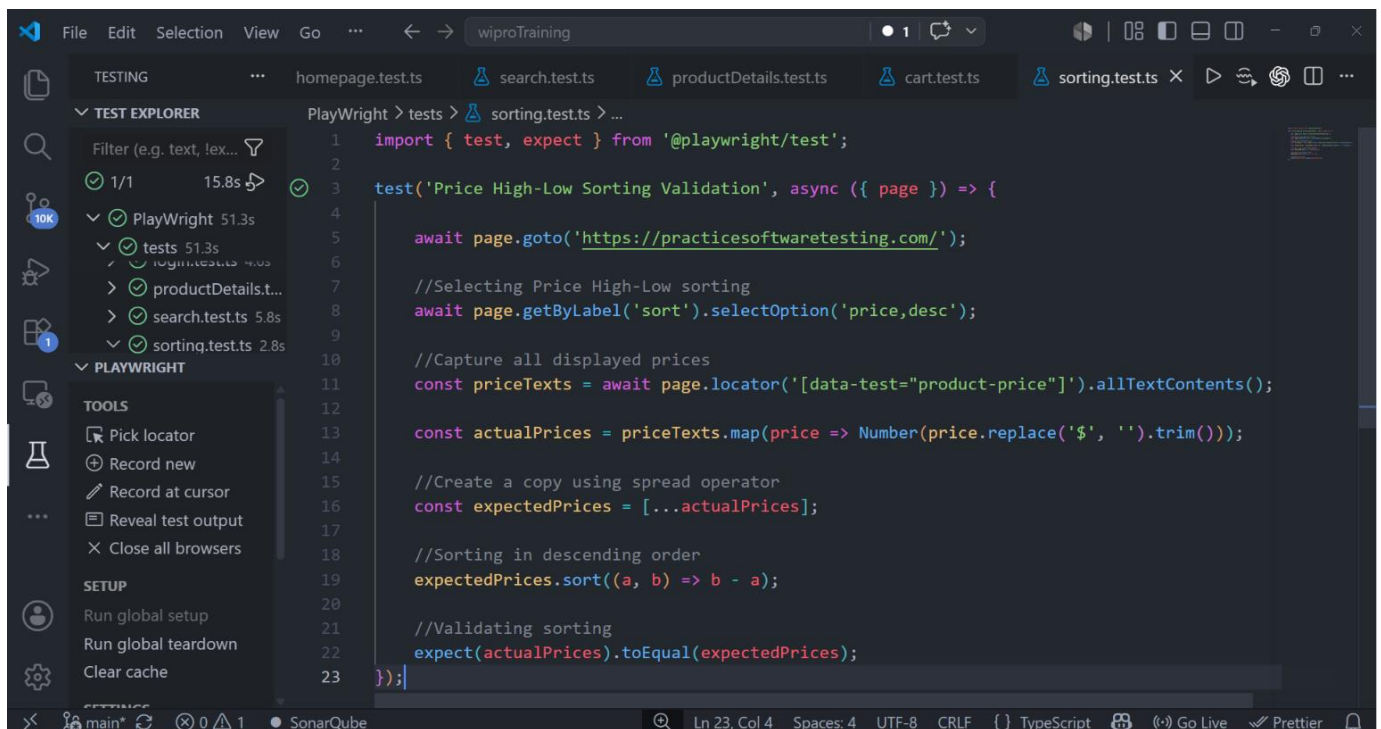
Options:

- Name A-Z
- Name Z-A
- Price Low-High
- Price High-Low

Participants MUST:

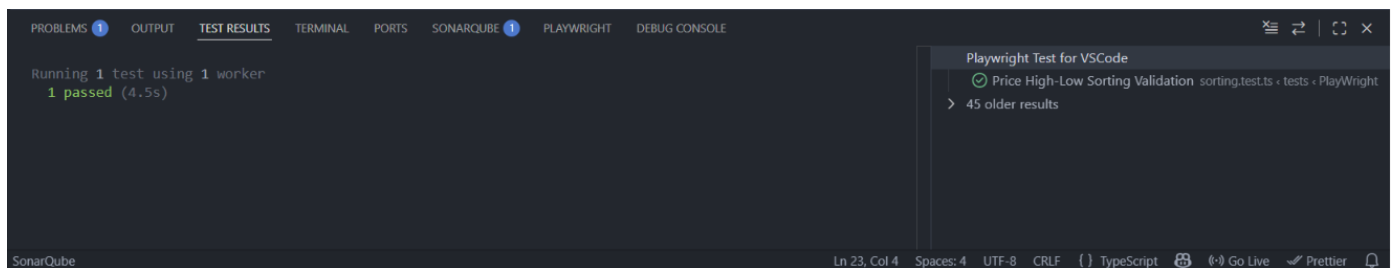
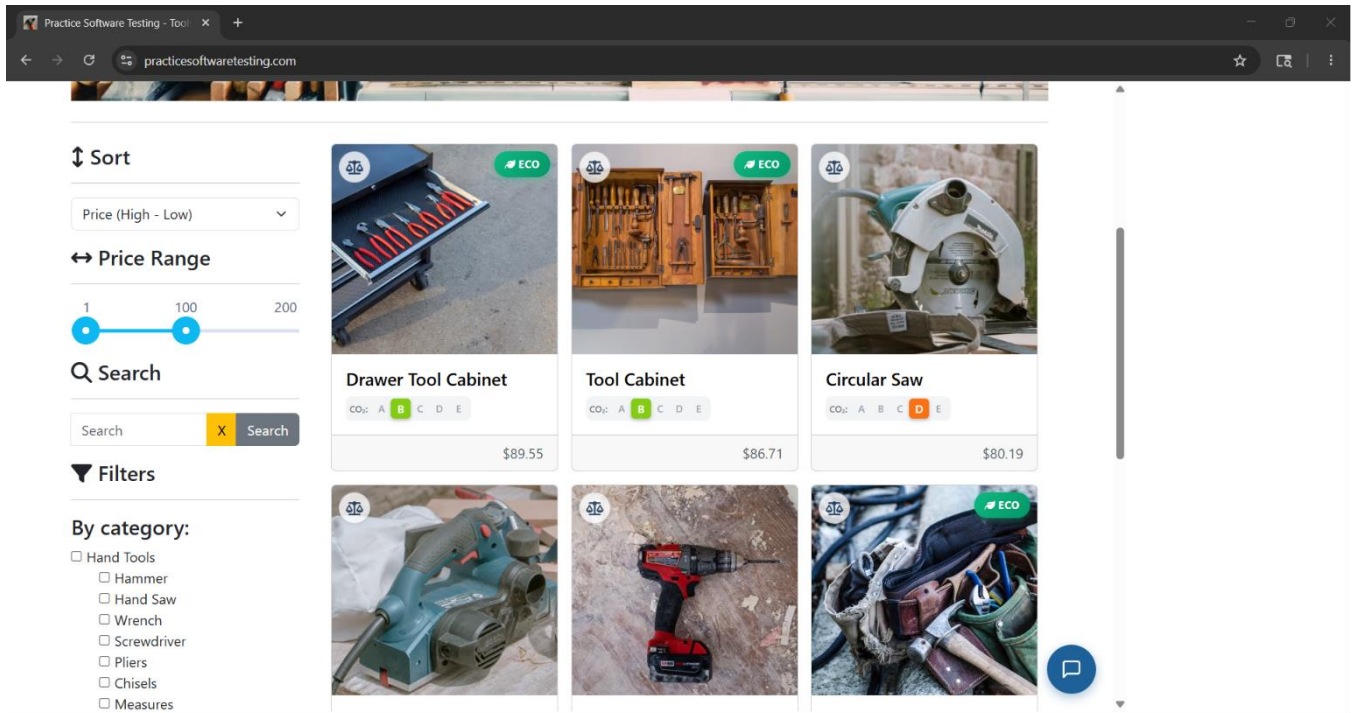
- capture displayed values dynamically
- compare actual sorted order
- validate sorting logic properly

Screenshot of Code:



```
1 import { test, expect } from '@playwright/test';
2
3 test('Price High-Low Sorting Validation', async ({ page }) => {
4
5     await page.goto('https://practicesoftwaretesting.com/');
6
7     //Selecting Price High-Low sorting
8     await page.getByLabel('sort').selectOption('price,desc');
9
10    //Capture all displayed prices
11    const priceTexts = await page.locator('[data-test="product-price"]').allTextContents();
12
13    const actualPrices = priceTexts.map(price => Number(price.replace('$', ' ').trim()));
14
15    //Create a copy using spread operator
16    const expectedPrices = [...actualPrices];
17
18    //Sorting in descending order
19    expectedPrices.sort((a, b) => b - a);
20
21    //Validating sorting
22    expect(actualPrices).toEqual(expectedPrices);
23 });
```


Screenshot of Output/Terminal:



Task 6 - Negative Login Scenario

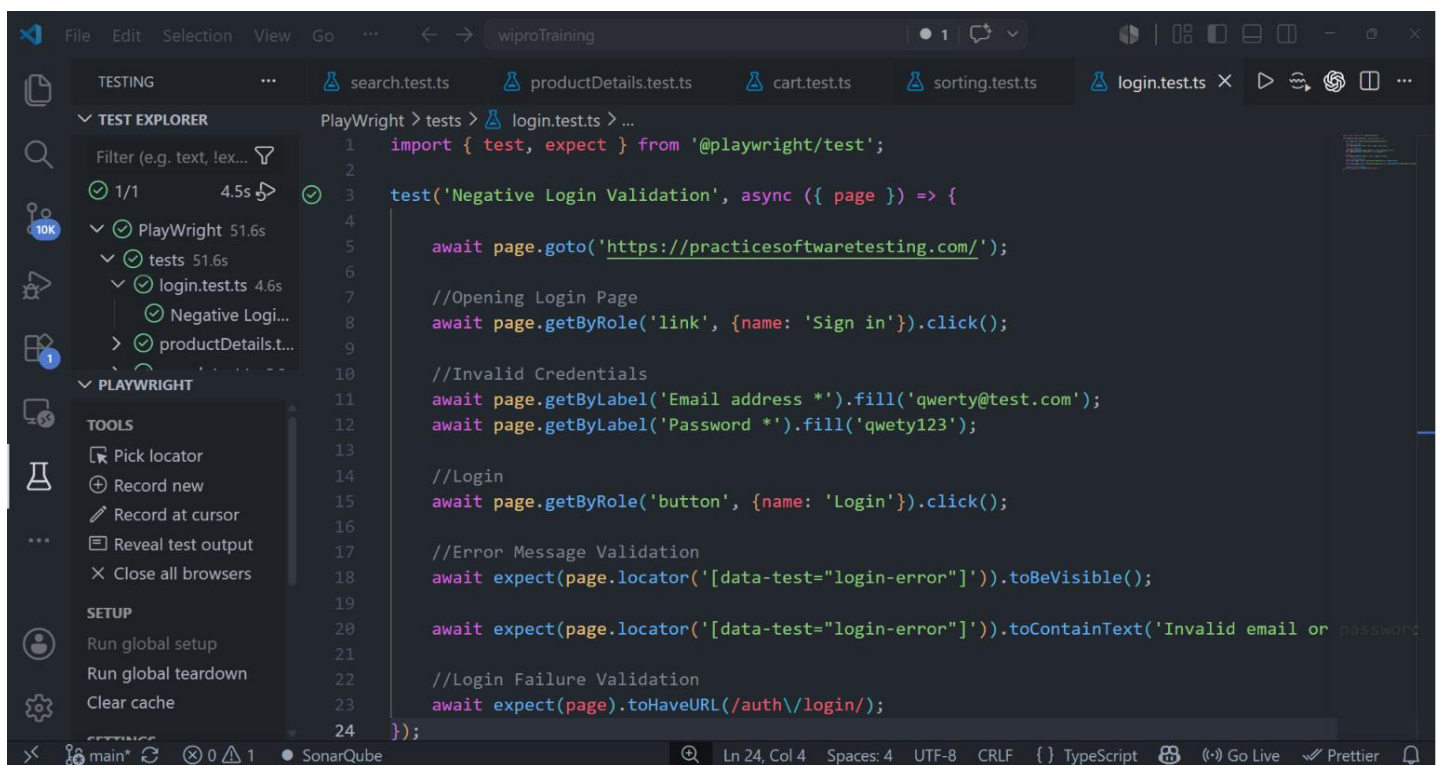
Requirements

Attempt login using invalid credentials.

Validate:

- error message visibility
- login failure behavior

Screenshot of Code:

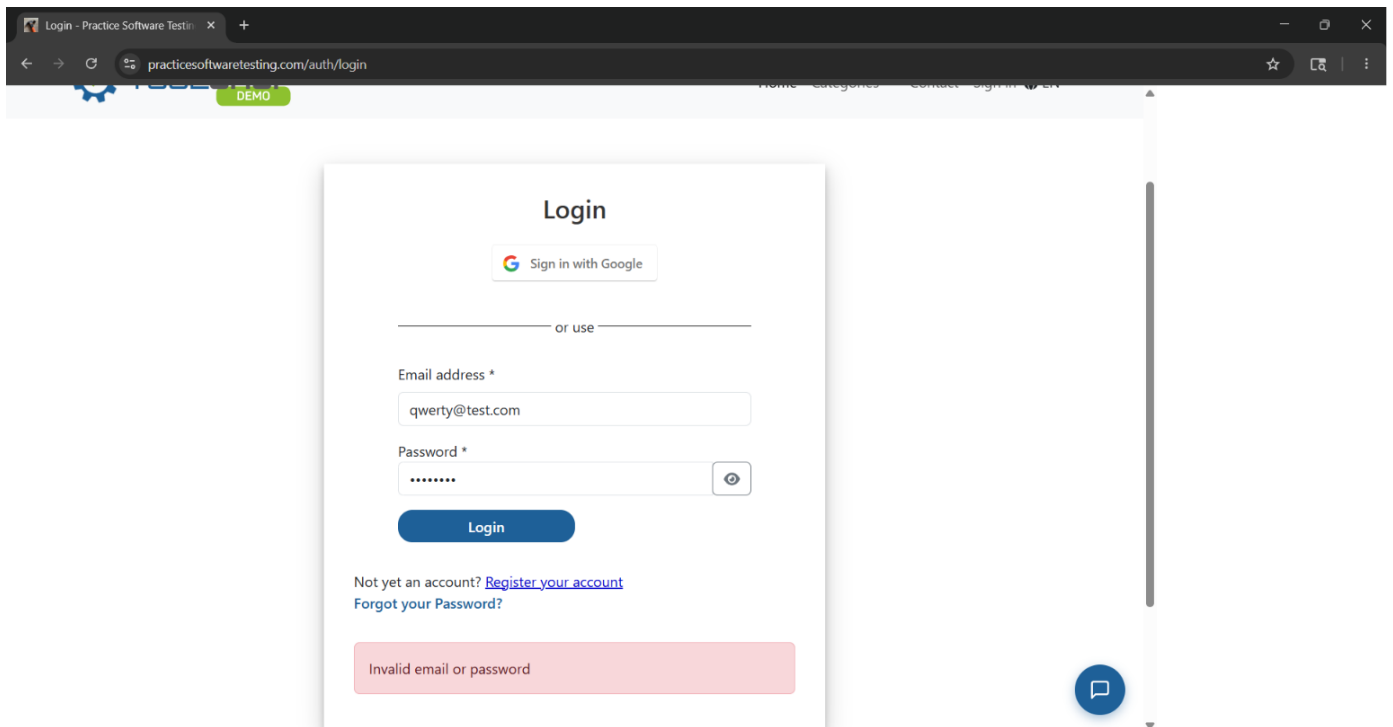


The screenshot shows a Visual Studio Code editor window with a Playwright test file named `login.test.ts` open. The file is located in the `tests` directory under `login.test.ts`. The test is titled `Negative Login Validation` and is an asynchronous function that takes a `page` object as an argument. The test performs the following steps:

- Imports `test` and `expect` from `@playwright/test`.
- Opens the login page by clicking on a link with the name `Sign in`.
- Enters invalid credentials: `qwerty@test.com` for the email address and `qwerty123` for the password.
- Clicks the `Login` button.
- Validates that an error message is visible by checking for the presence of an element with the data-test attribute `login-error`.
- Validates that the error message contains the text `Invalid email or password`.
- Validates that the URL is `/auth/login/`.

```
1 import { test, expect } from '@playwright/test';
2
3 test('Negative Login Validation', async ({ page }) => {
4
5     await page.goto('https://practicesoftwaretesting.com/');
6
7     //Opening Login Page
8     await page.getByRole('link', {name: 'Sign in'}).click();
9
10    //Invalid Credentials
11    await page.getByLabel('Email address *').fill('qwerty@test.com');
12    await page.getByLabel('Password *').fill('qwerty123');
13
14    //Login
15    await page.getByRole('button', {name: 'Login'}).click();
16
17    //Error Message Validation
18    await expect(page.locator('[data-test="login-error"]')).toBeVisible();
19
20    await expect(page.locator('[data-test="login-error"]')).toContainText('Invalid email or password');
21
22    //Login Failure Validation
23    await expect(page).toHaveURL(/auth/login/);
24 });
```

Screenshot of Output/Terminal:



Task 7 - Checkout Flow

Requirements

Complete a purchase flow.

Flow:

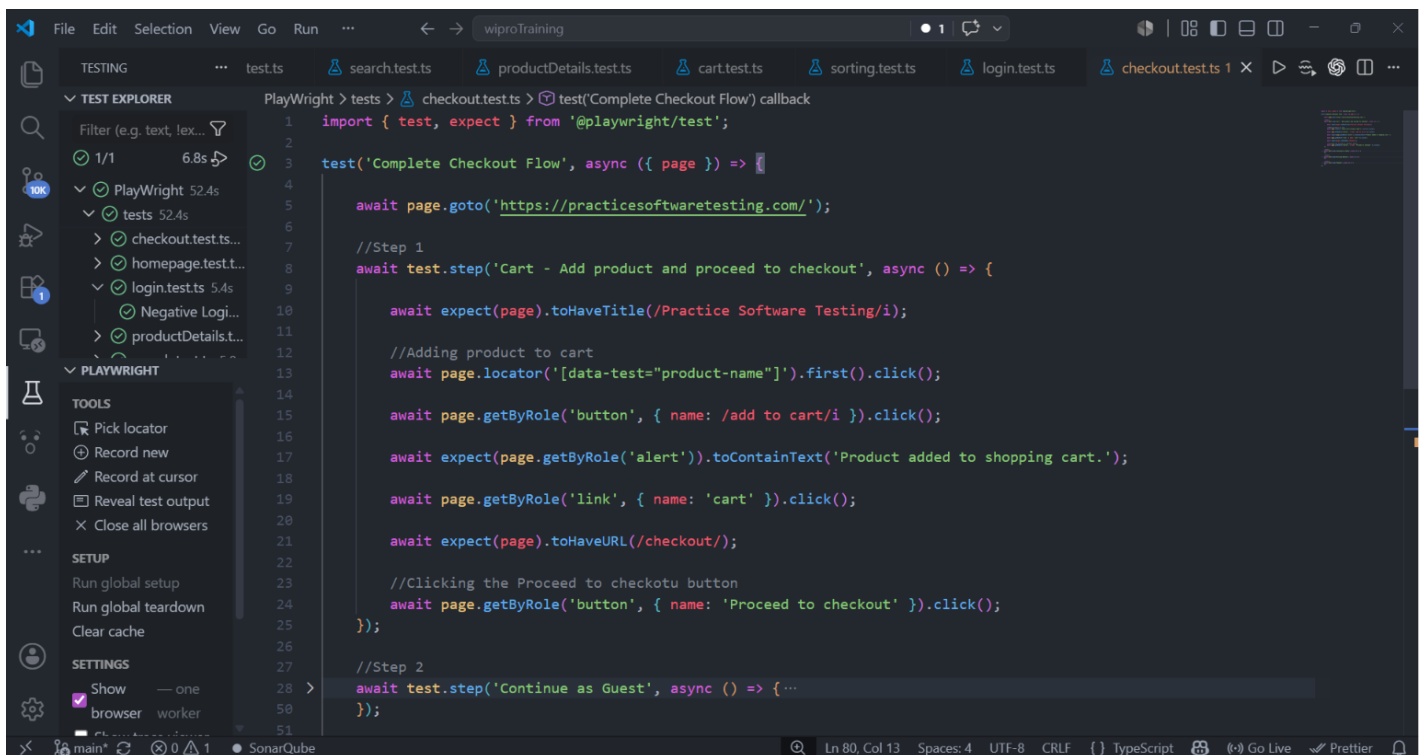
- add products
- proceed to cart
- checkout
- fill required details
- place order

Validate:

- successful order placement message
- final confirmation page

Screenshot of Code:

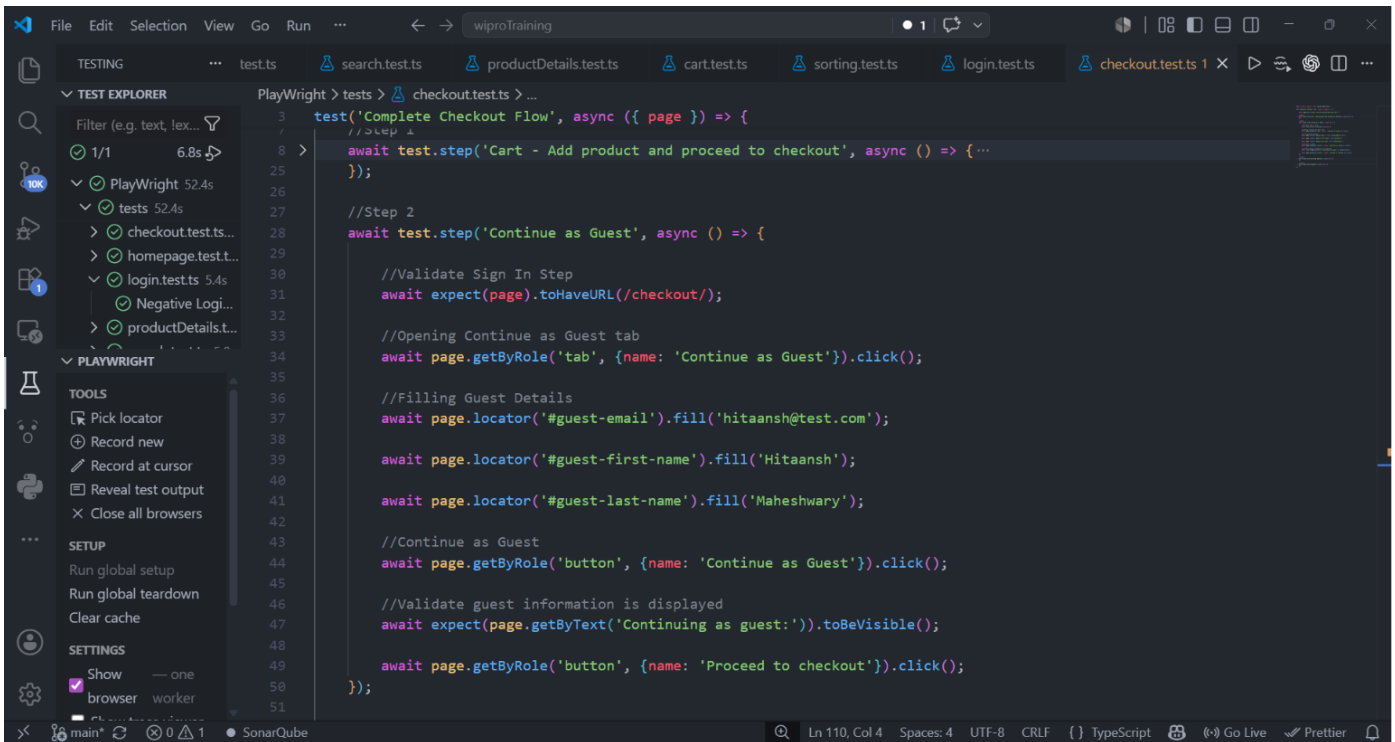
Step1:



The screenshot shows a Visual Studio Code editor window with a Playwright test file named `checkout.test.ts`. The test is titled "Complete Checkout Flow" and is an async function. It starts by navigating to `https://practicesoftwaretesting.com/`. The first step, "Cart - Add product and proceed to checkout", involves clicking a product name, clicking the "add to cart" button, verifying the alert text "Product added to shopping cart.", clicking the "cart" link, verifying the URL is `/checkout/`, and clicking the "Proceed to checkout" button. The second step, "Continue as Guest", is partially visible. The editor interface includes a sidebar with the Test Explorer and Playwright toolbars, and a status bar at the bottom showing file encoding and formatting settings.

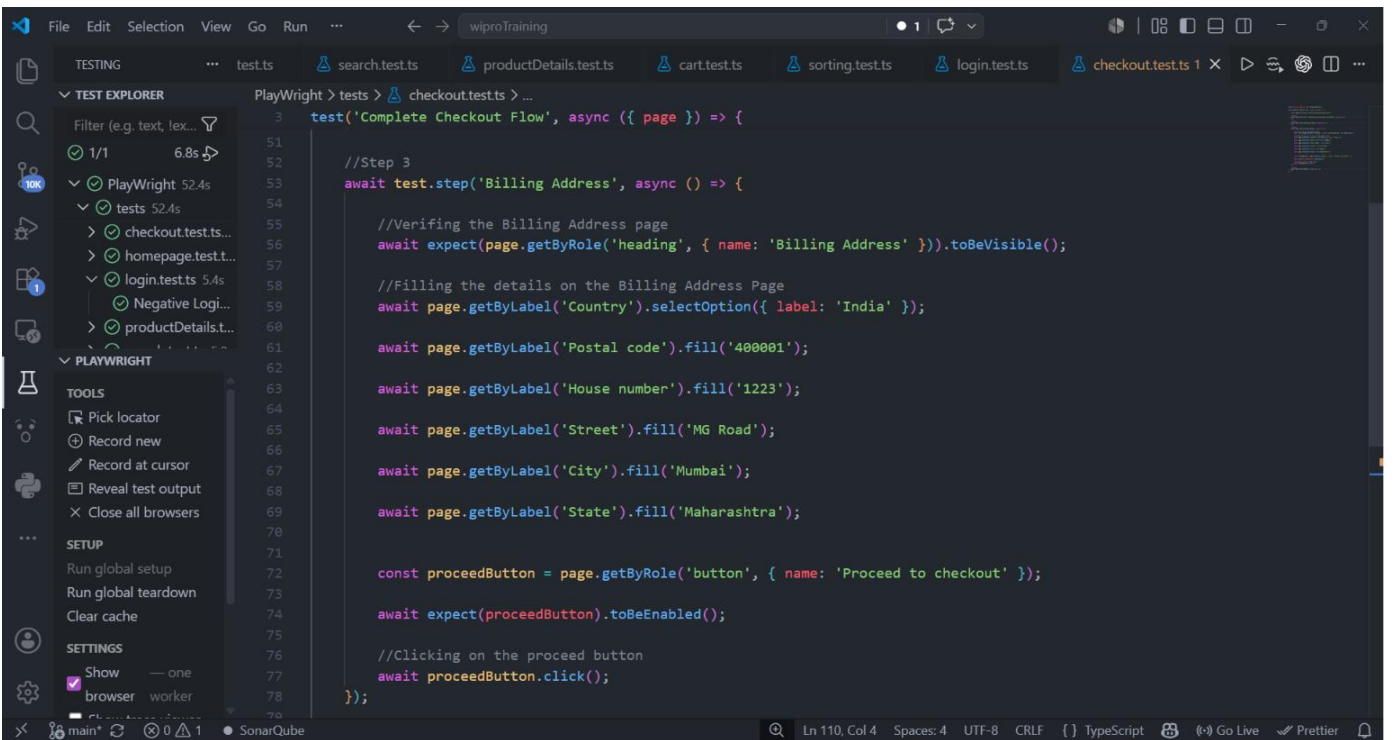
```
1 import { test, expect } from '@playwright/test';
2
3 test('Complete Checkout Flow', async ({ page }) => {
4
5     await page.goto('https://practicesoftwaretesting.com/');
6
7     //Step 1
8     await test.step('Cart - Add product and proceed to checkout', async () => {
9
10         await expect(page).toHaveTitle(/Practice Software Testing/i);
11
12         //Adding product to cart
13         await page.locator('[data-test="product-name"]').first().click();
14
15         await page.getByRole('button', { name: /add to cart/i }).click();
16
17         await expect(page.getByRole('alert')).toContainText('Product added to shopping cart.');
```

Step2:



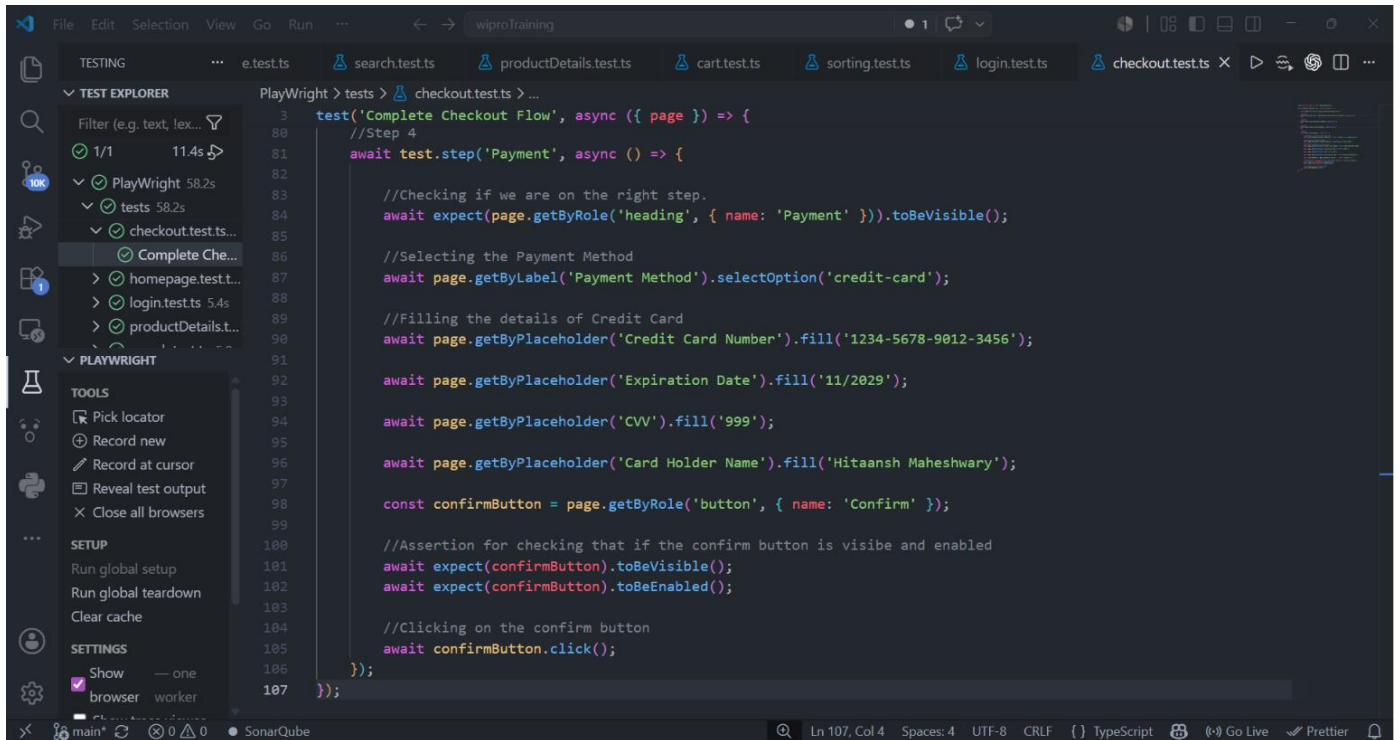
```
3  test('Complete Checkout Flow', async ({ page }) => {
4    //Step 1
5    await test.step('Cart - Add product and proceed to checkout', async () => { ...
6  });
7
8  //Step 2
9  await test.step('Continue as Guest', async () => {
10
11    //Validate Sign In Step
12    await expect(page).toHaveURL(/checkout/);
13
14    //Opening Continue as Guest tab
15    await page.getByRole('tab', {name: 'Continue as Guest'}).click();
16
17    //Filling Guest Details
18    await page.locator('#guest-email').fill('hitaansh@test.com');
19
20    await page.locator('#guest-first-name').fill('Hitaansh');
21
22    await page.locator('#guest-last-name').fill('Maheshwary');
23
24    //Continue as Guest
25    await page.getByRole('button', {name: 'Continue as Guest'}).click();
26
27    //Validate guest information is displayed
28    await expect(page.getByText('Continuing as guest:')).toBeVisible();
29
30    await page.getByRole('button', {name: 'Proceed to checkout'}).click();
31  });
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
```

Step3:



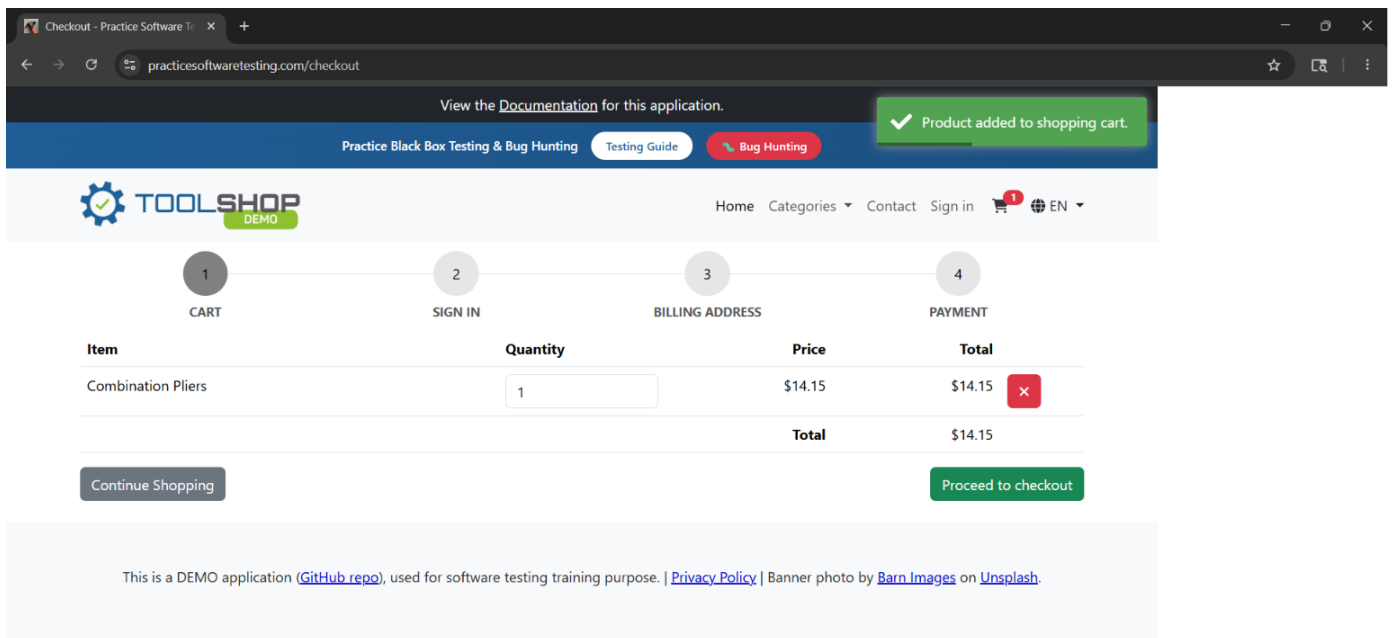
```
3  test('Complete Checkout Flow', async ({ page }) => {
4    //Step 1
5    await test.step('Cart - Add product and proceed to checkout', async () => { ...
6  });
7
8  //Step 2
9  await test.step('Continue as Guest', async () => {
10
11    //Validate Sign In Step
12    await expect(page).toHaveURL(/checkout/);
13
14    //Opening Continue as Guest tab
15    await page.getByRole('tab', {name: 'Continue as Guest'}).click();
16
17    //Filling Guest Details
18    await page.locator('#guest-email').fill('hitaansh@test.com');
19
20    await page.locator('#guest-first-name').fill('Hitaansh');
21
22    await page.locator('#guest-last-name').fill('Maheshwary');
23
24    //Continue as Guest
25    await page.getByRole('button', {name: 'Continue as Guest'}).click();
26
27    //Validate guest information is displayed
28    await expect(page.getByText('Continuing as guest:')).toBeVisible();
29
30    await page.getByRole('button', {name: 'Proceed to checkout'}).click();
31  });
32
33  //Step 3
34  await test.step('Billing Address', async () => {
35
36    //Verifying the Billing Address page
37    await expect(page.getByRole('heading', { name: 'Billing Address' })).toBeVisible();
38
39    //Filling the details on the Billing Address Page
40    await page.getByLabel('Country').selectOption({ label: 'India' });
41
42    await page.getByLabel('Postal code').fill('400001');
43
44    await page.getByLabel('House number').fill('1223');
45
46    await page.getByLabel('Street').fill('MG Road');
47
48    await page.getByLabel('City').fill('Mumbai');
49
50    await page.getByLabel('State').fill('Maharashtra');
51
52    const proceedButton = page.getByRole('button', { name: 'Proceed to checkout' });
53
54    await expect(proceedButton).toBeEnabled();
55
56    //Clicking on the proceed button
57    await proceedButton.click();
58  });
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
```

Step4:



```
3  test('Complete Checkout Flow', async ({ page }) => {
80  //Step 4
81  await test.step('Payment', async () => {
82
83      //Checking if we are on the right step.
84      await expect(page.getByRole('heading', { name: 'Payment' })).toBeVisible();
85
86      //Selecting the Payment Method
87      await page.getByLabel('Payment Method').selectOption('credit-card');
88
89      //Filling the details of Credit Card
90      await page.getByPlaceholder('Credit Card Number').fill('1234-5678-9012-3456');
91
92      await page.getByPlaceholder('Expiration Date').fill('11/2029');
93
94      await page.getByPlaceholder('CVV').fill('999');
95
96      await page.getByPlaceholder('Card Holder Name').fill('Hitaansh Maheshwary');
97
98      const confirmButton = page.getByRole('button', { name: 'Confirm' });
99
100     //Assertion for checking that if the confirm button is visible and enabled
101     await expect(confirmButton).toBeVisible();
102     await expect(confirmButton).toBeEnabled();
103
104     //Clicking on the confirm button
105     await confirmButton.click();
106
107 });
```

Screenshot of Output/Terminal:



Checkout - Practice Software T... x +

practicesoftwaretesting.com/checkout

Home Categories Contact Sign in 1 EN

1 CART 2 SIGN IN 3 BILLING ADDRESS 4 PAYMENT

[Sign in](#) [Continue as Guest](#)

Continue as Guest

Checkout without creating an account

Email address *

hitaansh@test.com

First name *

Hitaansh

Last name *

Maheshwary

[Continue as Guest](#)

This is a DEMO application ([GitHub repo](#)), used for software testing training purpose. | [Privacy Policy](#) | Banner photo by [Barn Images](#) on [Unsplash](#)

Checkout - Practice Software T... x +

practicesoftwaretesting.com/checkout

Home Categories Contact Sign in 1 EN

1 CART 2 SIGN IN 3 BILLING ADDRESS 4 PAYMENT

Billing Address

Country

India

Enter country, postal code and house number. We will fill in the rest automatically.

Postal code

400001

House number

1223

Street

Metz Ridges

City

East Theresa

State

Vermont

[Proceed to checkout](#)

